



UT03: Análisis Exploratorio de Datos (EDA)

El camino esencial hacia la comprensión profunda de cualquier conjunto de datos

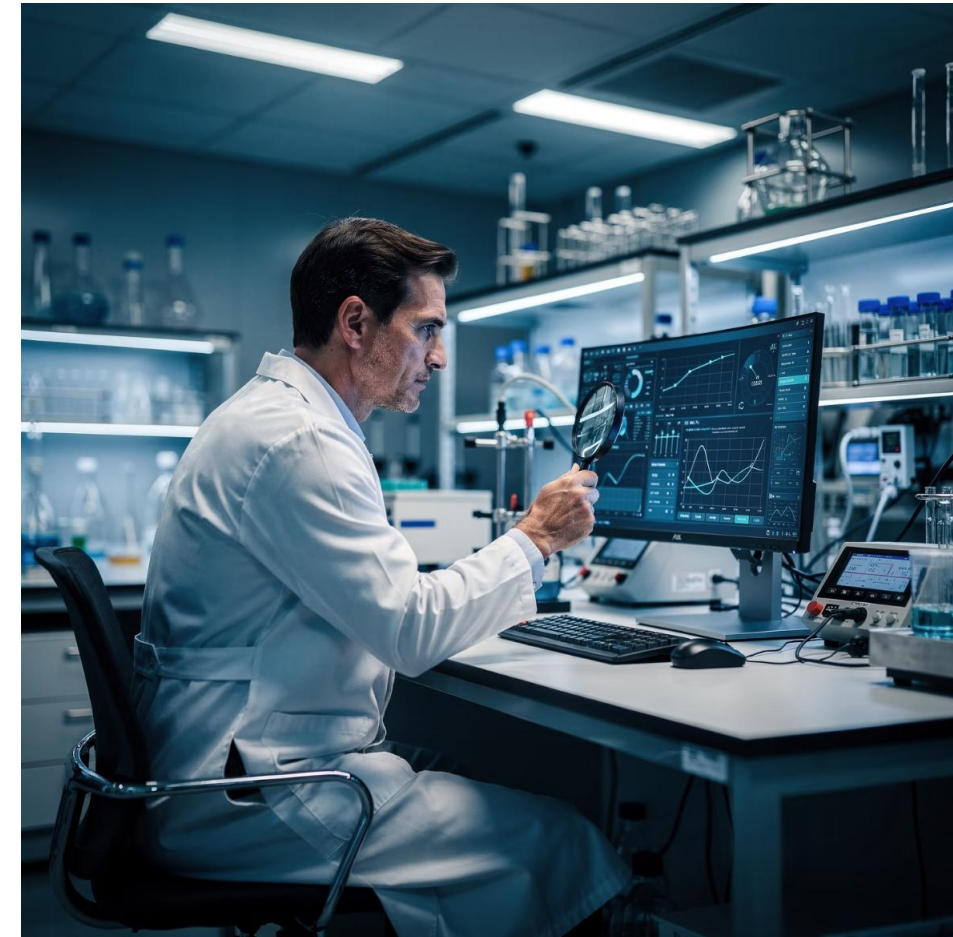
FUNDAMENTOS

¿Qué es el Análisis Exploratorio de Datos?

El **Análisis Exploratorio de Datos (EDA)** representa la primera y más crucial tarea que desempeña un científico de datos al recibir un conjunto de información. No se trata simplemente de ejecutar código automático, sino de un proceso investigativo donde el analista se convierte en detective de los datos.

Este análisis consiste en revisar sistemáticamente los datos para comprender "de qué tratan", vislumbrar posibles **patrones ocultos** y reconocer distribuciones estadísticas que pueden resultar útiles para análisis posteriores.

El objetivo principal del EDA es triple: primero, sacar conclusiones rápidas sobre la viabilidad de un proyecto; segundo, evaluar la calidad y consistencia de los datos disponibles; y tercero, formular hipótesis preliminares antes de aplicar modelos estadísticos o de machine learning más avanzados.



❏ **Dato clave:** Un EDA bien ejecutado puede ahorrar semanas de trabajo posterior al identificar problemas de calidad de datos desde el principio.

El Detective de Datos: Una Analogía Práctica

Hacer un **EDA** es como ser un **detective en la escena del crimen**. Antes de acusar a nadie (hacer una predicción con Machine Learning), primero debes investigar meticulosamente cada detalle.

01

Observar el escenario completo

Ver el tamaño del dataset con `.shape` para entender la magnitud del caso

02

Tomar notas generales

Usar `.describe()` para obtener promedios y estadísticas básicas de lo que hay

03

Hacer preguntas a los testigos

Filtrar datos con `.groupby()` para interrogar segmentos específicos

04

Buscar huellas y pistas extrañas

Detectar outliers o patrones visuales anómalos con gráficos

05

Identificar cómplices

Ver si dos variables suelen ir siempre juntas usando `.corr()`

Si el detective no limpia la escena o ignora pistas contradictorias (datos nulos o erróneos), su conclusión final será inevitablemente falsa. Del mismo modo, un EDA descuidado conduce a modelos predictivos defectuosos.



CAPÍTULO 1

Resumen de Grandes Volúmenes de Datos

Para facilitar la toma de decisiones estratégicas, es absolutamente necesario transformar datos brutos y masivos en resúmenes informativos, comprensibles y accionables. Python y pandas proporcionan herramientas poderosas para esta tarea fundamental.

Exploración Estructural: Conociendo el Terreno

Dimensiones del Dataset

Antes de analizar cualquier contenido, debemos conocer el **volumen exacto** de los datos con los que trabajamos. El atributo `.shape` nos devuelve una tupla con el número de filas (observaciones) y columnas (variables).

```
df.shape  
# Resultado: (768, 9)  
# 768 pacientes, 9 variables
```

Esta información es crítica porque nos indica si estamos ante un dataset pequeño (cientos de filas), mediano (miles) o grande (millones), lo cual influirá en las técnicas de análisis y visualización que podamos aplicar.

Listado de Variables

El método `.columns.tolist()` permite obtener una lista legible de todas las columnas disponibles en el DataFrame, facilitando la comprensión de qué información tenemos a nuestra disposición.

```
df.columns.tolist()  
# ['Edad', 'Glucosa', 'Presion',  
#  'IMC', 'Insulina', ...]
```

Conocer los nombres exactos es esencial para el análisis posterior, ya que nos permite identificar qué variables son relevantes para nuestras preguntas de investigación.

Inspección Visual Rápida de los Datos

1

Método `.head()`

Visualiza las primeras 5 filas (por defecto) de la tabla, permitiendo ver el formato de los datos, el tipo de información en cada columna y detectar posibles inconsistencias en el inicio del dataset.

```
df.head(10) # Ver las primeras 10 filas
```



Consejo práctico: Siempre revisa tanto el inicio como el final de tus datos. A veces los problemas de calidad se concentran en zonas específicas del dataset.

2

Método `.tail()`

Muestra las últimas 5 filas del DataFrame. Esto es particularmente útil para verificar si los datos al final del conjunto mantienen la misma estructura y calidad que al principio, o si hay cambios inesperados.

```
df.tail(10) # Ver las últimas 10 filas
```

	Trg	Piso	Metas	Porcentaje
142418	9.6.50	87.50	17.10	2.96
718.45	6.0.12	10.30	15.80	2.96
425.30	101.80	40.55	17.25	2.74
671.70	0.5.60	25.41	7.44	1.96
617.25	1.1.28	46.50	2.58	1.96
873.40	1.1.97	27.40	12.96	2.96
70.70	9.40	16.30	17.78	12.10
134.87	9.5%	97.77	16.26	12.10
909.85	5.4.97	95.27	15.10	12.10
615.38	3.9.97	45.00	15.10	1.74
582.39	1.1.93	112.06	12.25	2.74
355.20	2.3.00	111.97	9.89	1.96
810.19	2.8.97	132.45	12.25	2.74
622.25	8.8.07	10.11	1.96	1.74
969.99	3.8.53	69.50	12.25	1.96
	2.8.90	71.90	9.24	2.10
	1.01	88.20	9.96	1.96
		22.63	11.13	2.96

Estadística Descriptiva: El Corazón del EDA

El método `.describe()` es la herramienta fundamental y más utilizada para resumir datos numéricos de forma automática. Esta función calcula y devuelve un conjunto completo de estadísticos descriptivos que proporcionan una visión panorámica de cada variable numérica.

Información que proporciona:

- **count:** Número de valores no nulos (válidos) en la columna
- **mean:** Media aritmética de todos los valores
- **std:** Desviación estándar (dispersión respecto a la media)
- **min:** Valor mínimo observado
- **25%, 50%, 75%:** Percentiles que dividen los datos en cuartiles
- **max:** Valor máximo observado

```
df.describe()
```

#	Edad	Glucosa	IMC
# count	768.0	768.0	768.0
# mean	33.2	120.9	31.9
# std	11.8	31.9	7.9
# min	21.0	44.0	18.2
# 25%	24.0	99.0	27.3
# 50%	29.0	117.0	32.0
# 75%	41.0	140.2	36.6
# max	81.0	199.0	67.1

Este resumen permite identificar rápidamente valores atípicos, rangos de variación y la distribución general de cada variable.

Diagnóstico de Tipos de Datos



Método .info()

Proporciona un resumen completo del DataFrame incluyendo el número de entradas, tipos de datos de cada columna, cantidad de valores no nulos y uso de memoria. Ideal para una visión general rápida.

```
df.info()  
# Int64, Float64, Object...
```



Atributo .dtypes

Devuelve específicamente el tipo de dato de cada columna sin información adicional. Útil cuando solo necesitas verificar si las variables son texto (object), enteros (int64) o decimales (float64).

```
df.dtypes  
# Edad: int64  
# Nombre: object
```

Verificar los tipos de datos es crucial porque determina qué operaciones son posibles. Por ejemplo, no puedes calcular la media de una columna de texto, y fechas almacenadas como strings no permiten cálculos temporales hasta convertirlas al tipo datetime.



Formulación y Respuesta a Preguntas Relevantes

El analista debe pensar de forma crítica y no solo ejecutar código mecánicamente. Las preguntas deben orientarse al dominio del problema específico: salud, ventas, educación, finanzas, etc.

Un científico de datos experto no se limita a aplicar fórmulas predefinidas, sino que formula **preguntas inteligentes** que revelan insights ocultos en los datos. La calidad de las respuestas depende directamente de la calidad de las preguntas formuladas.

Técnicas de Segmentación de Datos

Filtrado Booleano

Para responder preguntas sobre grupos específicos, se utiliza el filtrado mediante condiciones lógicas. Esta técnica permite aislar subconjuntos de datos que cumplen ciertos criterios.

```
# Pacientes con glucosa alta
diabeticos = df[df["Glucosa"] > 140]

# Mujeres mayores de 40 años
mujeres_40 = df[(df["Edad"] > 40) &
                 (df["Sexo"] == "F")]
```

El filtrado puede combinar múltiples condiciones usando operadores lógicos como `&` (AND), `|` (OR) y `~` (NOT).

Método `.groupby()`

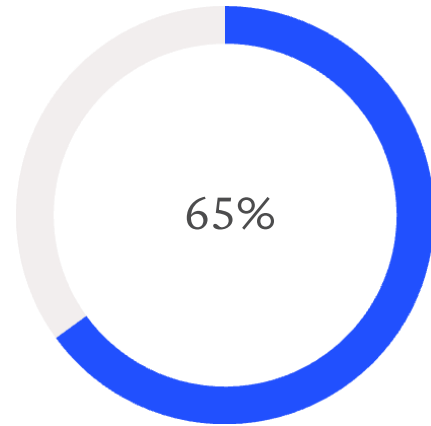
Agrupar datos según categorías y permite calcular estadísticas por grupo. Es la herramienta perfecta para comparaciones entre segmentos.

```
# Glucosa promedio por rango de edad
df.groupby("Rango_Edad")["Glucosa"].mean()

# Edad: 20-30 -> 115.2
# Edad: 31-40 -> 122.8
# Edad: 41-50 -> 128.5
```

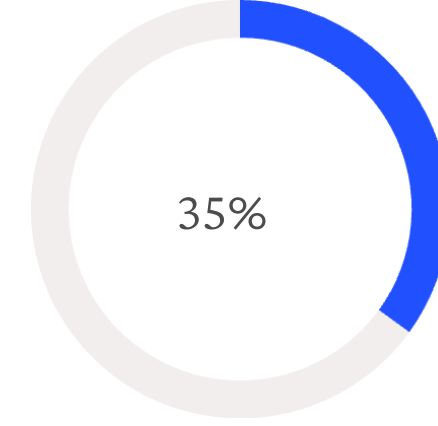
Puedes aplicar múltiples funciones agregadas simultáneamente: `.agg(['mean', 'std', 'count'])`

Cálculo de Proporciones y Frecuencias



No Diabéticos

Proporción de pacientes sin diagnóstico de diabetes en el estudio



Diabéticos

Proporción de pacientes diagnosticados con diabetes

La función `.value_counts(normalize=True)` es esencial para determinar porcentajes y frecuencias de categorías. Este método cuenta las ocurrencias únicas en una columna y puede expresarlas como proporciones.

```
# Frecuencias absolutas
df["Resultado"].value_counts()

# Frecuencias relativas (%)
df["Resultado"].value_counts(
    normalize=True
)
```



Comprender las proporciones es fundamental en ciencia de datos porque revela el balance de clases en problemas de clasificación, identifica categorías minoritarias y ayuda a detectar posibles sesgos en los datos de entrenamiento.

Creación de Columnas Personalizadas

A menudo, la respuesta a nuestras preguntas analíticas no está directamente en los datos originales, sino en las **relaciones matemáticas** entre diferentes variables. Python permite crear nuevas columnas derivadas mediante operaciones sobre columnas existentes.



Datos Originales

Insulina: 85 µU/ml
Glucosa: 120 mg/dl



Operación

División: Insulina / Glucosa



Nueva Variable

Ratio Insulina/Glucosa: 0.708

```
# Crear ratio personalizado
df["Ratio_IG"] = df["Insulina"] / df["Glucosa"]

# Categorizar IMC
df["Categoria_IMC"] = df["IMC"].apply(
    lambda x: "Bajo" if x < 18.5
    else "Normal" if x < 25
    else "Sobrepeso" if x < 30
    else "Obesidad"
)
```

Estas transformaciones permiten capturar interacciones complejas entre variables que los modelos simples podrían no detectar. El arte del feature engineering (ingeniería de características) reside precisamente en crear variables derivadas que mejoren el poder predictivo.



CAPÍTULO 3

Reconocimiento de Patrones Significativos

Los patrones en los datos permiten identificar **tendencias sistemáticas**, **comportamientos anómalos** y **estructuras ocultas** que pueden ser cruciales para la toma de decisiones. Un patrón no es solo una repetición, sino una regularidad que aporta información predictiva o explicativa sobre el fenómeno estudiado.

Distribuciones y Frecuencias: Visualización Clave



Histogramas

Permiten ver dónde se concentran los valores (por ejemplo, los niveles de glucosa más comunes en una población) y revelan la forma de la distribución: simétrica, asimétrica, unimodal o bimodal.

También muestran si hay colas largas que indican presencia de valores extremos.



Gráficos de Tarta

Útiles para comparar visualmente el peso relativo de cada categoría respecto al total. Funcionan mejor con pocas categorías (3-6) y cuando queremos enfatizar proporciones.

No son ideales para comparaciones precisas de valores similares.



Gráficos de Barras

Excelentes para comparar frecuencias entre categorías. A diferencia de las tartas, permiten ordenar categorías por valor y facilitan la comparación de múltiples variables lado a lado.

Son más precisos cuando los valores son cercanos.

Detección de Outliers: Los Valores Atípicos

Los **outliers** o valores atípicos son observaciones que se alejan drásticamente del resto de los datos. Pueden representar errores de medición, casos excepcionales reales o indicadores de fraude.

Método del Rango Intercuartílico (IQR)

```
Q1 = df["Glucosa"].quantile(0.25)
Q3 = df["Glucosa"].quantile(0.75)
IQR = Q3 - Q1

limite_inf = Q1 - 1.5 * IQR
limite_sup = Q3 + 1.5 * IQR

outliers = df[
    (df["Glucosa"] < limite_inf) |
    (df["Glucosa"] > limite_sup)
]
```

Método de Desviaciones Estándar

```
media = df["Glucosa"].mean()
std = df["Glucosa"].std()

outliers = df[
    abs(df["Glucosa"] - media) > 3 * std
]
```

Los outliers no siempre deben eliminarse. Primero hay que investigar su origen: ¿son errores o información valiosa? En medicina, un valor extremo puede indicar una condición crítica que requiere atención.

📌 **Regla práctica:** Valores más allá de ± 3 desviaciones estándar representan menos del 0.3% de los datos en distribuciones normales.

Discretización (Binning): Agrupar para Simplificar

La discretización mediante `pd.cut()` permite transformar variables continuas en categorías o intervalos, facilitando el análisis de tendencias por grupos y simplificando la interpretación de resultados.

```
# Crear grupos de edad
df["Grupo_Edad"] = pd.cut(
    df["Edad"],
    bins=[20, 30, 40, 50, 60, 70, 80],
    labels=["20-29", "30-39", "40-49",
           "50-59", "60-69", "70-79"]
)

# Analizar glucosa por grupo
df.groupby("Grupo_Edad")["Glucosa"].mean()
```

Esta técnica es especialmente poderosa para observar si ciertas condiciones médicas se agravan con el tiempo o si patrones de comportamiento cambian según rangos de edad, ingreso, etc.



115

Grupo 20-29
Glucosa media (mg/dl)

122

Grupo 30-39
Glucosa media (mg/dl)

131

Grupo 40-49
Glucosa media (mg/dl)

CAPÍTULO 4

Identificación de Correlaciones

La correlación mide la fuerza y dirección de la relación lineal entre dos variables numéricas. Es una de las herramientas más poderosas del EDA para descubrir asociaciones entre variables.

El coeficiente de correlación de Pearson varía entre -1 y +1, donde valores cercanos a +1 indican correlación positiva fuerte, valores cercanos a -1 indican correlación negativa fuerte, y valores cercanos a 0 indican ausencia de correlación lineal.



Cálculo e Interpretación de Correlaciones

Cálculo en Python

El método `.corr()` aplicado sobre un DataFrame genera automáticamente una matriz de correlación que muestra la correlación entre cada par de variables numéricas.

```
matriz_corr = df.corr()
print(matriz_corr)

# Correlación específica entre dos variables
corr_edad_glucosa = df["Edad"].corr(
    df["Glucosa"]
)
print(f"Correlación: {corr_edad_glucosa:.3f}")
```

Tipos de Correlación

Positiva (+0.7 a +1.0)

Las variables crecen juntas. Ejemplo: Edad y número de Embarazos típicamente aumentan juntos.

Negativa (-1.0 a -0.7)

Una variable crece mientras la otra decrece. Ejemplo: Actividad física e IMC a menudo muestran correlación negativa.

Cercana a Cero (-0.3 a +0.3)

No hay relación lineal clara. Ejemplo: IMC y número de Embarazos generalmente no están relacionados.

Visualización de Correlaciones

Las **nubes de puntos (Scatter plots)** son la mejor forma de confirmar visualmente una correlación y detectar si existen tendencias ocultas, relaciones no lineales o grupos de datos con comportamientos diferentes.

```
import matplotlib.pyplot as plt

plt.scatter(df["IMC"],
            df["Glucosa"], alpha=0.5)
plt.xlabel("IMC (Índice de Masa Corporal)")
plt.ylabel("Glucosa (mg/dl)")
plt.title("Relación entre IMC y Glucosa")
plt.show()
```

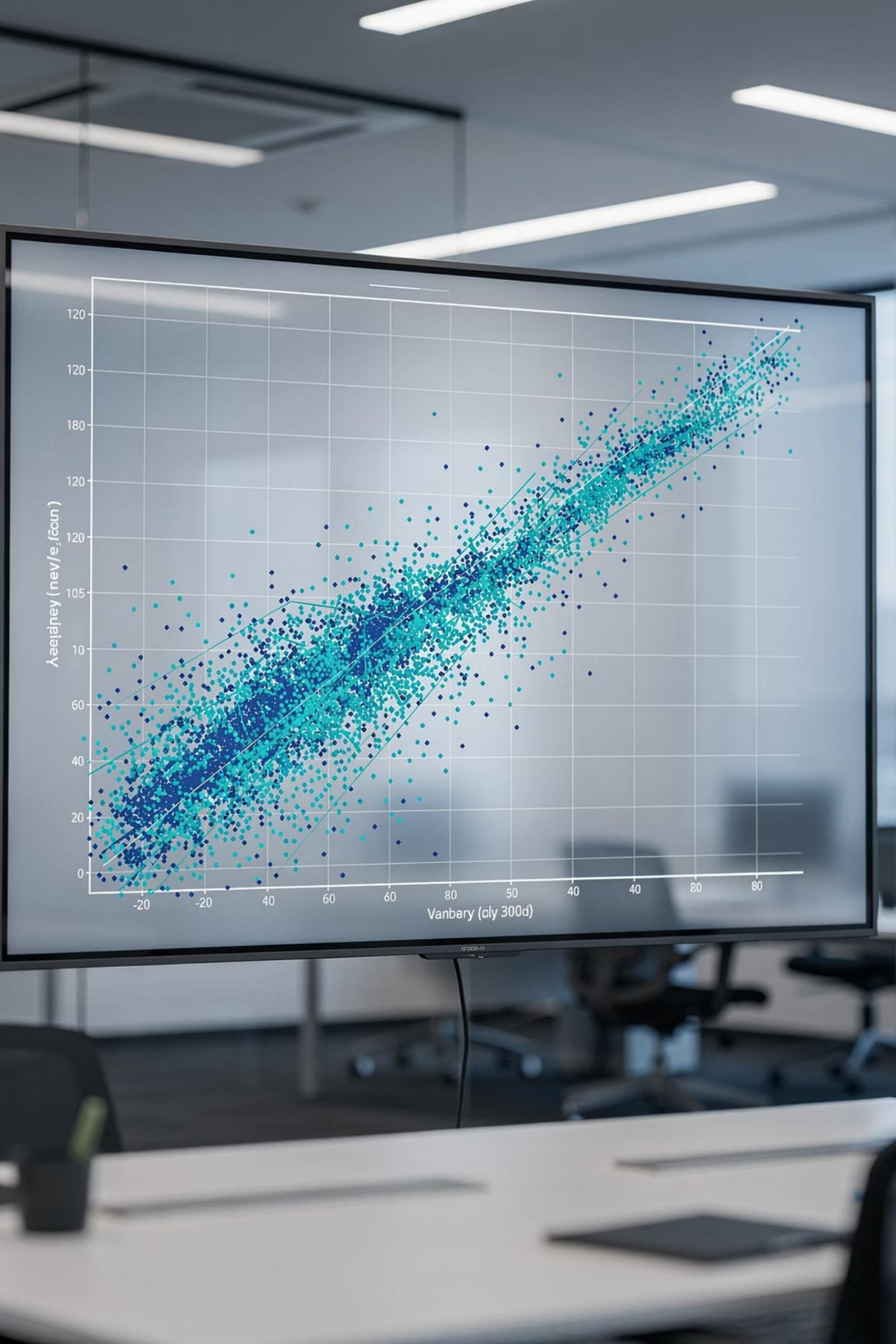
Los mapas de calor (heatmaps) son ideales para visualizar matrices de correlación completas, usando colores para representar la intensidad de la correlación entre múltiples variables simultáneamente.

```
import seaborn as sns

sns.heatmap(df.corr(),
            annot=True,
            cmap="coolwarm",
            center=0,
            vmin=-1, vmax=1)

plt.title("Matriz de Correlación")
plt.show()
```

❏ **Advertencia crucial:** Correlación NO implica causalidad. Dos variables pueden estar correlacionadas por casualidad o porque ambas dependen de una tercera variable no observada.



Conclusiones: Dominando el Arte del EDA

El Análisis Exploratorio de Datos no es simplemente una fase preliminar del análisis, sino el **fundamento sobre el cual se construye todo proyecto exitoso** de ciencia de datos. Un EDA bien ejecutado puede prevenir errores costosos, revelar insights inesperados y orientar decisiones estratégicas informadas.

Sistemático y Exhaustivo

No te saltes pasos. Cada herramienta del EDA (shape, describe, info, correlaciones) aporta una pieza única del rompecabezas completo.

Pensamiento Crítico

El código ejecuta operaciones, pero tú debes interpretar resultados, cuestionar anomalías y formular hipótesis inteligentes basadas en el dominio del problema.

Visualización Constante

Los gráficos no son opcionales: son esenciales. Revelan patrones que las tablas numéricas ocultan y comunican hallazgos de forma más efectiva.

Documentación Clara

Registra tus decisiones, hallazgos y suposiciones. Un EDA sin documentación es conocimiento perdido para tu equipo y para tu yo futuro.

"El análisis de datos es un arte tanto como una ciencia. La maestría viene de la práctica constante, la curiosidad incansable y el compromiso con la verdad que los datos revelan." - *Principio fundamental del científico de datos*